Modifying `sort()` Behavior

The built-in function `sort` accepts a variety of predefined constants (as a second argument) that will modify sorting behavior. We'll pick three: `SORT_REGULAR` (the default), `SORT_NUMERIC`, and `SORT_STRING`. While we're at it, let's bring together some of the techniques we've seen throughout the chapter.

```
$sortFlags = [
    'regular' => SORT_REGULAR,
    'numeric' => SORT_NUMERIC,
    'string' => SORT_STRING,
];

$data =[
    'regular' => ['a', '-2', '-1', '-5.7', '-57', '0', '1'],
    'numeric' => ['a', '-2', '-1', '-5.7', '-57', '0', '1'],
    'string' => ['a', '-2', '-1', '-5.7', '-57', '0', '1'],
];

foreach ($data as $flag => $arr) {
    sort($arr, $sortFlags[$flag]);

    print "{$flag}: \t";

    foreach ($arr as $ele) {
        print "{$ele}\t";
    }

    print "\n";
}

Output:
regular:    -57 -5.7    -2  -1  0   1   a
numeric:    -57 -5.7    -2  -1  a   0   1
string:     -1  -2  -5.7    -57 0   1   a
```

We're working with a set of negative and positive numbers, and a letter has also been thrown into the set to demonstrate how type coercion affects sorting and comparison. The keys of each sub-array of `$data` are used as a map to the sort flag we wish to use